Java Best Practices

Core Java, OOP, collections, streams, exceptions, and maintainable code

Purpose: This document is a sample knowledge source for the DevPilot AI RAG ingestion pipeline. It is written with headings and concise sections so the service can extract text, create chunks, embed them, and retrieve relevant passages.

1. Java Language Fundamentals

Java is a statically typed, class-based language. Source code is compiled to bytecode, which the Java Virtual Machine executes. The JVM implementation provides runtime services such as memory management and garbage collection.

JDK, JRE, and JVM

JVM executes bytecode. The runtime environment supplies the JVM and runtime libraries. The JDK adds developer tools such as the compiler and diagnostic utilities.

Primitives and references

Primitive types include boolean, byte, short, int, long, char, float, and double. Reference variables refer to objects or arrays. Use appropriate types and validate values at system boundaries.

2. Object-Oriented Programming

Concept	Meaning	Example
Encapsulation	Protect internal state behind methods	private fields and methods
Inheritance	Reuse or specialize a parent type	class SavingsAccount extends Account
Polymorphism	Use one interface for multiple implementations	PaymentService
Abstraction	Expose essential behavior and hide details	interface ExpenseRepository

Prefer composition over inheritance when a relationship is not truly an 'is-a' relationship. Keep classes focused on one clear responsibility.

3. Collections and Generics

List preserves order and permits duplicates; Set models uniqueness; Map associates keys with values. Choose the implementation based on the required behavior and performance characteristics.

```
List<String> names = List.of("Asha", "Mina");\nMap<String, Integer> counts = new HashMap<>();\ncou
nts.put("expense", 3);
```

Generics provide compile-time type safety and reduce casts. Avoid using raw collection types in application code.

4. Exceptions and Resource Management

Checked exceptions must be caught or declared. Unchecked exceptions extend RuntimeException and commonly represent programming errors or invalid state. Catch exceptions at boundaries where you can add context, recover, or translate them.

Use try-with-resources for resources that implement AutoCloseable, such as streams and JDBC resources, so they are closed reliably.

5. Streams and Lambdas

Streams express data processing pipelines such as filter, map, sorted, and collect. Intermediate operations are generally lazy; a terminal operation triggers evaluation. Avoid side effects in stream operations.

```
List<String> expensiveItems = expenses.stream()\n    .filter(e -> e.amount().compareTo(limit) > 0)
\n    .map(Expense::description)\n    .toList();
```

Use a loop when it is clearer than a stream. Parallel streams are not automatically faster and require care with shared state and blocking operations.

6. Equality, Immutability, and Concurrency

Use equals to test logical equality and hashCode consistently when objects are keys in hash-based collections. Immutable objects are easier to reason about because their state cannot change after construction.

Concurrency requires safe publication and coordination of shared mutable state. Prefer immutable data, thread-safe abstractions, and well-defined ownership over ad hoc synchronization.

7. Java Best Practices Checklist

- Choose descriptive names and keep methods focused.
- Prefer interfaces at useful boundaries, not for every class automatically.
- Use BigDecimal for monetary values rather than binary floating-point types.
- Validate external input and handle errors intentionally.
- Avoid swallowing exceptions or logging secrets.
- Write tests for edge cases and failure behavior.

8. RAG Questions

- What is the difference between an interface and an abstract class?
- When should you use List, Set, or Map?
- Why is BigDecimal suitable for currency calculations?
- What makes stream intermediate operations lazy?
- How should equals and hashCode relate to each other?

End of document. Use this file for development and testing; verify all examples against your actual application configuration before treating them as production standards.